SIMATS ENGINEERING

Course: Object-Oriented Programming using C++ (DSA01)

ASSIGNMENT 1 [CO1]

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Namma Metro Smart-Card Fare System

Student Name	**Syed Nazma**
Register Number	192425193
Department / Section	AI&ML
Compiler and version	g++ 13.3.0 (Ubuntu), -std=c++17
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file
Submission date	07-11-2026
GitHub repository link	https://github.com/nazma824/DSA0117

Contents

- Task 1 - Understand the Problem and Identify the Objects
- Task 2 - Tokens, Keywords, Identifiers and Constants
- Task 3 - Data Types and Type Compatibility
- Task 4 - Implicit Conversions (with Predict the Output)
- Task 5 - Expressions and Precedence (with Predict the Output)
- Task 6 - Control Structures (with Complete the Code)
- Task 7 - From Procedural to Object-Oriented
- Task 8 - Read and Print Data Safely
- Task 9 - Implement and Test the Fare Logic
- Task 10 - Compare Expectations with Execution
- Task 11 - Recommend and Defend the Final Design
- Task 12 - Day-4 Design Challenge
- Appendix - GitHub Submission Guide

Problem Summary and Common Assumptions

A city metro has three zones (Inner, Middle, Outer). The fare is Rs 10, Rs 20 or Rs 30 by the farthest zone travelled, with a 20% surcharge in peak hours (8-10 AM and 5-8 PM). A journey is allowed only if at least Rs 50 remains on the card after the fare. Students and senior citizens receive a concession. The first software had paisa errors, an inconsistent peak check (10 AM and 8 PM), a negative balance and fare values typed in several places.

Assumptions used throughout this report

- All money is held as an integer number of paise (Rs 1 = 100 paise).
- Peak hours are half-open intervals: 08:00 to 09:59 and 17:00 to 19:59. So 10:00 and 20:00 are always off-peak.
- Concession (not stated in the assignment): Student 50%, Senior citizen 30%, applied after the peak surcharge. This can be changed in one constant.
- Airport zone fare is Rs 60 (Task 12). Weekend discount is 10% and replaces the peak surcharge on weekends.
- Every program in this report was compiled with g++ 13.3.0 using -std=c++17 -Wall -Wextra and run. The outputs shown are the actual outputs.

Task 1 - Understand the Problem and Identify the Objects

1.1 Objects, data and behaviour

Object	Data	Behaviour	Why it is an object
SmartCard	balance (paise), card id	canTravel(), deduct(), recharge()	It owns the money and must protect the rule that the balance never falls below Rs 50 after a journey.
Passenger	name, category (Regular, Student, Senior)	concessionPercent()	The concession depends on who the person is, not on the trip.
Journey	entry zone, exit zone, hour, weekend flag	farthestZone(), isPeak()	It groups all facts needed to price one trip.
Zone	Inner, Middle, Outer (later Airport)	none (enum class values)	A fixed set of named values; a type of its own prevents mixing zones with plain numbers.
FareTable	base fare per zone, surcharge %, minimum balance	baseFare(zone)	Keeps every fare number in one place so a revision is made once.

1.2 UML class diagram

SmartCard
- balance_ : long long - pass_ : const Pass*
+ canTravel(f) : bool + deduct(f) : bool + recharge(a) : bool + balance() : long long

Journey
- zone : Zone - hour : int - weekend : bool
+ isPeak() : bool + fare() : long long

FareTable
- FARE_PAISE[] : long long - PEAK_PERCENT : int - MIN_BALANCE : long long
+ baseFare(z) : long long

Passenger
- name : string - category : Category
+ concessionPercent() : int

Pass (Task 12)
- a_ : Zone - b_ : Zone
+ covers(z) : bool

Zone <<enum class>>
Inner Middle Outer

Airport
(no methods)

Relationships: a Passenger owns one SmartCard (association). A SmartCard may hold zero or one Pass (aggregation). A Journey uses a Zone and a FareTable to compute the fare. StudentPassenger and SeniorPassenger inherit from Passenger (generalisation) and override concessionPercent().

1.3 Procedural style versus object-oriented style

In a procedural style we would keep global variables such as balance and fares[] and free functions such as computeFare() and deduct(). Because the data is global, any function can change balance, so the negative-balance bug could come from anywhere in the program, and the fare values are likely to be typed in many places. In the object-oriented style the data and the functions that protect it are packaged together. The balance is private and only deduct() changes it, after checking the rule. The rule lives in one place, and a bug can only come from one class. The program then reads like the real system: a card pays for a journey.

1.4 OOP principles and where they appear

- **Encapsulation:** SmartCard keeps balance_ private; only deduct() and recharge() can change it.
- **Abstraction:** gate code calls card.deduct(fare) without knowing how the Rs 50 rule is checked.
- **Inheritance:** StudentPassenger and SeniorPassenger derive from Passenger; a MonthlyPass could derive from a common travel-authority base class.
- **Polymorphism:** a virtual concessionPercent() returns 0, 50 or 30 depending on the passenger type, so the fare code does not need an if-chain on category.

1.5 Rules every correct journey must satisfy

1. The balance after payment is at least 5000 paise (Rs 50). This also makes a negative balance impossible. Justification: it is the central business rule and the cause of the reported bug.
2. Fare = base fare of the farthest zone, plus 20% only in peak hours. Justification: this is the pricing rule given in the scenario.

3. Peak is 08:00 up to but not including 10:00, and 17:00 up to but not including 20:00. Justification: half-open intervals remove the 10 AM / 8 PM inconsistency.
4. All money is exact integer paise. Justification: removes the "off by a paisa" errors.
5. The zone and hour must be valid and a recharge must be positive. Justification: rejects invalid input before it can corrupt the balance.

1.6 Alternative concept and complexity analysis: purely procedural solution

```
long long balance = 10000;                // global
const long long fares[] = {1000, 2000, 3000};
long long computeFare(int zone, bool peak);
void deduct(long long f) { balance -= f; }   // no check at all
```

Time: one journey needs a few comparisons and one subtraction: O(1) in both styles. **Space:** one balance value per card: O(1) in both styles. The real difference is the cost of change. When the minimum balance changes, the procedural program needs every place that touches balance to be found and edited, which is O(c) where c is the number of call sites. In the object-oriented program only SmartCard::canTravel (or one constant) changes, which is O(1) places. The object-oriented style therefore wins on maintainability even though speed and memory are the same.

Evidence: objects table, UML diagram, rules and assumptions above.

Task 2 - Tokens, Keywords, Identifiers and Constants

2.1 The five kinds of tokens

Consider the line: constexpr int PEAK_SURCHARGE_PERCENT = 20;

Token kind	Example from this line	Other examples in the program
Keyword	constexpr, int	enum, class, switch, return, if, for
Identifier	PEAK_SURCHARGE_PERCENT	fare, canTravel, balance_, Zone
Constant (literal)	20	1000, 5000, '0', true
Operator	=	+, -, *, /, %, &&, ||, <, >=, ?:
Punctuator	;	{ } ( ) [ ] , ::

2.2 #define, const and constexpr

- **#define PEAK 20** is text replacement done by the preprocessor. It has no type, no scope and no debugger symbol.
- **const int PEAK = 20;** has a type and a scope, but its value may be decided at run time.
- **constexpr int PEAK = 20;** is guaranteed to be known at compile time, so it can be used as an array size or a switch label. It is preferred for the fare table because it is type-checked, scoped and has no run-time cost.

2.3 enum and enum class

A plain enum puts its names into the surrounding scope (Inner becomes a global name) and converts silently to int. An enum class is scoped (Zone::Inner) and does not convert implicitly, so int x = Zone::Inner; is a compile-time error. This stops a zone from being mixed up with an ordinary number, so enum class is safer.

2.4 Identifier naming rules and conventions

- Rules: letters, digits and underscore only; cannot start with a digit; cannot be a keyword; case-sensitive.
- Conventions used: constants in UPPER_SNAKE_CASE, classes in PascalCase, functions and variables in camelCase, private data members with a trailing underscore.

2.5 Evidence: one place for all constants

```
enum class Zone { Inner, Middle, Outer };
constexpr int       BASE_PAISE[] = {1000, 2000, 3000};  // Rs 10, 20, 30
constexpr int       PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;           // Rs 50
```

A fare revision is now made by editing this one block and recompiling.

2.6 Alternative concept and complexity analysis: #define macros

```
#define BASE_INNER 1000
#define BASE_MIDDLE 2000
#define PEAK_PERCENT 20
```

With macros there is no type checking (BASE_INNER can silently be used where a zone is expected), no scope (the name clashes with any identically named variable everywhere after the #define), and the debugger shows only the number. **Memory:** a macro occupies no storage; a constexpr value usually occupies none either, because the compiler folds it into the code. **Time:** both are O(1) and the running time is unaffected. The constexpr/enum class choice gives the same speed and memory with much better safety.

Task 3 - Data Types and Type Compatibility

3.1 Data-type table

Value	Type chosen	Reason	sizeof on my system
Fare, balance, totals	long long (paise)	exact integers, huge range	8 bytes
Hour (0-23)	int	small whole number	4 bytes
Peak / weekend flag	bool	two states	1 byte
Zone	enum class Zone	named fixed set	typically 4 bytes
Percent values	int	whole numbers	4 bytes
Journeys per day	long long	up to 3,000,000 and beyond	8 bytes
Separator / letter	char	one character	1 byte

Sizes and limits measured with sizeof and <climits> (program t3.cpp):

Output:

```
char 1 bool 1 int 4 long long 8 float 4 double 8 unsigned 4
INT_MAX 2147483647 INT_MIN -2147483648 UINT_MAX 4294967295 LLONG_MAX
9223372036854775807
```

3.2 Why integer paise and not double rupees

Values such as 0.1 cannot be stored exactly in binary floating point, so repeated additions drift. Money must be exact, so the program counts whole paise in an integer type.

3.3 Type compatibility

In an expression that mixes bool, int and double, the compiler applies the usual arithmetic conversions: bool is promoted to int, and int is converted to double, so the result is double. Narrowing back (double to int) discards the fractional part silently unless brace initialisation is used.

3.4 Is int enough for a day's collection?

For the whole network, 3,000,000 journeys at Rs 20.10 each is 6,030,000,000 paise, which is larger than INT_MAX (2,147,483,647). So int is not enough; long long (maximum about 9.2 x 10^18) is.

3.5 Evidence: precision problem and its resolution

```
double rs = 0; long long paise = 0;
for (long long i = 0; i < n; ++i) { rs += 0.10; paise += 10; }
std::cout << std::fixed << std::setprecision(10) << rs << ' ' << paise / 100.0;
```

Output:

```
1000 fares:     double=100.0000000000       exact=100.0000000000       error=-0.0000000000
100000 fares:   double=10000.0000000188 exact=10000.0000000000   error=0.0000000188
1000000 fares:  double=100000.0000013329   exact=100000.0000000000   error=0.0000013329
10000000 fares: double=999999.9998389754   exact=1000000.0000000000  error=-
0.0001610246
```

The error in the double total is zero for small n but becomes visible as n grows, while the integer-paise total is exact at every size. **Resolution:** store money as long long paise.

3.6 Alternative concept and complexity analysis: double rupees

If money were stored as double rupees, each fare would be a number like 20.10 and the totals would be built by repeated floating-point addition. **Time per addition:** O(1) in both. **Space per value:** 8 bytes in both. **Accuracy:** the measurements above show the error is not zero and its size varies with n (about 1.9 x 10^-8 at 100,000 fares, 1.3 x 10^-6 at 1,000,000 and 1.6 x 10^-4 at 10,000,000), so the worst-case accumulated error grows roughly in proportion to n, O(n x epsilon). The integer-paise version has zero error, so it is preferred.

Task 4 - Implicit Conversions

4.1 Where C++ converts without being told

- **Integer division then widening:** double c = a / b; divides two ints first (7 / 2 = 3), then converts 3 to 3.0.
- **Signed against unsigned:** in s < u the signed value is converted to unsigned, so -1 becomes 4294967295; the compiler warns with -Wsign-compare.
- **Narrowing:** int x = 3.7; silently stores 3. Brace initialisation int x{3.7}; is rejected at compile time.
- **static_cast:** needed when you intentionally want a different type, e.g. static_cast<double>(a) / b for real division.

4.2 Predict the output

My predicted output (written before running):

```
3 3.5
C 68
1 true
s >= u
```

4.3 Actual output (g++ 13.3.0, -std=c++17 -Wall -Wextra)

Output:

```
3 3.5
C 68
1 true
s >= u
```

Compiler warning received:

```
t4.cpp:13:17: warning: comparison of integer expressions of different
signedness: 'int' and 'unsigned int' [-Wsign-compare]
```

4.4 Explanation of each line

- **Line 1 (3 3.5):** a / b is integer division = 3, then 3 is converted to double 3.0 and printed as 3. d uses static_cast<double>(a) so the division is 7.0 / 2 = 3.5.

- **Line 2 (C 68):** 'A' + 2 is an int (67) converted back to char, which prints as C. ch + 1 promotes char to int (integer promotion), giving 68.
- **Line 3 (1 true):** an int 5 converts to bool true (any non-zero value). It prints as 1 until std::boolalpha is switched on, then as true.
- **Line 4 (s >= u):** s is converted to unsigned int (-1 becomes 4294967295), which is not less than 1, so s >= u is printed.

There was no difference between my prediction and the actual output. The only surprise was the warning on the signed/unsigned comparison.

4.5 Corrected code

```
int s{-1};
unsigned int u{1};
bool less = (s < 0) || (static_cast<unsigned int>(s) < u);  // correct: -1 < 1 is true
```

4.6 Alternative concept and complexity analysis: everything explicit

If every conversion were written with static_cast and brace initialisation were used everywhere, the generated machine code would be the same, so **running time and memory do not change** (O(1) per conversion). The benefit is safety: brace initialisation turns narrowing such as int x{3.7}; into a compile-time error instead of a silent wrong value at run time, and explicit casts make each conversion visible to the reader and to code review. The cost is more typing.

Task 5 - Expressions with the Right Precedence

5.1 Expressions for the fare logic

```
bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
long long f = base + (peak ? base * 20 / 100 : 0);
f -= f * concessionPercent / 100;       // compound assignment
```

- Precedence order used: * and / (and %) before + and -; relational operators (<, >=) before &&; && before ||; the conditional operator ?: and assignment operators are low and associate right to left.
- hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent to the parenthesised form because && binds tighter than ||. The parentheses are kept because they make the intent obvious and prevent mistakes.
- x++ gives the old value then increments; ++x increments then gives the new value. i = i++ is a mistake because it changes i twice in one expression without a defined order (undefined or unspecified behaviour in older C++).
- With negative operands / truncates toward zero and the sign of % follows the dividend: -10 / 3 = -3 and -10 % 3 = -1.

5.2 Predict the output

My predicted output (written before running):

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

5.3 Actual output

Output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

5.4 Line-by-line explanation

- **Line 1:** x + y * 2: * is applied first (3 * 2 = 6), then + (5 + 6 = 11). (x + y) * 2: the parentheses force + first (8), then * (16).
- **Line 2:** x++ + ++y: x++ yields 5 and makes x = 6; ++y makes y = 4 and yields 4; the + adds 5 + 4 = 9 and assigns to z (assignment is last). Printed: z=9, x=6, y=4.
- **Line 3:** x > y is 6 > 4, true, so ?: yields x = 6. x % 4 * 2: % and * have the same precedence and associate left to right, so (6 % 4) * 2 = 2 * 2 = 4.
- **Line 4:** 10 / 3 = 3, 10 % 3 = 1, -10 / 3 = -3 (truncation toward zero), -10 % 3 = -1 (sign of the dividend).
- **Line 5:** the loop adds i = 0, 3, 6, 9 (i += 3 each pass, stops when i reaches 12): 0 + 3 + 6 + 9 = 18.

There was no difference between my prediction and the actual output.

5.5 Alternative concept and complexity analysis: lookup table bool peak[24]

```
constexpr bool PEAK[24] = {
  false,false,false,false,false,false,false,false,  // 0-7
  true, true,                                       // 8, 9
  false,false,false,false,false,false,false,        // 10-16
  true, true, true,                                 // 17, 18, 19
  false,false,false,false};                         // 20-23
bool isPeak(int h) { return PEAK[h]; }              // h must be 0..23
```

Time: both are O(1). The expression performs up to four comparisons and two logical operations; the table performs one array index, so it needs fewer operations. **Space:** the table needs 24 bytes extra; the expression needs none. The table is easier to change (edit data, not logic) but the hour must be bounds-checked first, and an error in the 24 entries is harder to spot than an error in a short expression.

Task 6 - Control Structures

6.1 Choosing the control structure

- **if-else:** for ranges and true/false rules such as the peak check and the balance check.
- **switch:** for a fixed set of discrete values such as the zone; the compiler may build a jump table.
- **Conditional operator ?::** for a short either-or value, e.g. (peak ? surcharge : 0).
- **Loops:** a range-based for is best for processing every journey in an array (no index mistakes); a classic for when the index is needed; a while loop when the number of journeys is not known (read until input ends).
- **break and continue:** continue skips an invalid journey and moves to the next; break stops the loop when processing must end. Each switch case ends with break so execution does not fall through.
- **No case matches:** if no case matches and there is no default, the switch does nothing and the variable keeps its initial value (a silent wrong fare of 0). So a default that reports an invalid zone is added.

6.2 Pseudocode for the fare computation

```
ALGORITHM ComputeFare(zone, hour)
INPUT:  zone (Inner, Middle, Outer), hour (0 to 23)
OUTPUT: fare in paise, or -1 for invalid input
BEGIN
   IF hour < 0 OR hour > 23 THEN RETURN -1
   SWITCH zone
      CASE Inner:   base <- 1000
      CASE Middle:  base <- 2000
      CASE Outer:   base <- 3000
      DEFAULT:      RETURN -1
   END SWITCH
   IF (hour >= 8 AND hour < 10) OR (hour >= 17 AND hour < 20) THEN
      base <- base + (base * 20) / 100
   END IF
   RETURN base
END
```

```
ALGORITHM ProcessDay(journeys, balance)
BEGIN
   FOR EACH journey IN journeys DO
      f <- ComputeFare(journey.zone, journey.hour)
      IF f = -1 THEN CONTINUE
      IF balance - f >= 5000 THEN
         balance <- balance - f
      ELSE
         DISPLAY "Denied (low balance)"
      END IF
   END FOR
END
```

(Keyword capitalisation and arrows follow the SIMATS Engineering - Standard for Algorithmic Pseudocode; adjust if your handout uses different keywords.)

6.3 Complete the Code: TODO 1 and TODO 2

```
long long fare(Zone z, bool peak)
{
   long long base = 0;
   switch (z) {
      case Zone::Inner:  base = BASE_PAISE[0]; break;
      case Zone::Middle: base = BASE_PAISE[1]; break;
      case Zone::Outer:  base = BASE_PAISE[2]; break;
   }
   if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;  // integer arithmetic only
   return base;
}


bool canTravel(long long balance, long long f)
{
   return balance - f >= MIN_BALANCE_PAISE;
}
```

The rest of the program is exactly as given in the assignment.

Output:

```
Fare Rs 10.00 balance Rs 90
```

```
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

This matches the expected output. Check: Inner off-peak 1000, balance 9000. Outer peak 3000 + 600 = 3600, balance 5400. Middle 2000 would leave 3400, below 5000, so it is denied.

6.4 Boundary check at exactly 10 AM and 8 PM

```
bool isPeak(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }
// isPeak(8)=true  isPeak(10)=false  isPeak(17)=true  isPeak(20)=false
```

The half-open interval treats 10:00 and 20:00 as off-peak every time, so the earlier inconsistency cannot occur. These cases are also in the Task 9 test table and all pass.

6.5 Alternative concept and complexity analysis: array indexed by zone

```
constexpr long long FARE[] = {1000, 2000, 3000};
long long base = FARE[static_cast<int>(z)];
```

Time: both are O(1); a switch may be compiled into a jump table, an array lookup is one memory read. **Space:** the array uses 3 x 8 = 24 bytes; the switch stores the constants in the code. **Fourth zone:** the array needs one more element (and the enum one more value); the switch needs a new case in every switch that mentions zones, and a missed one silently gives a wrong fare. The array is easier to extend; the switch makes each zone explicit and lets the compiler warn about unhandled enum values.

Task 7 - From Procedural to Object-Oriented

7.1 Procedural version

```
#include <iostream>
long long balance = 10000;                    // global: anyone can change it
constexpr long long MIN_BALANCE = 5000;
bool canTravel(long long f) { return balance - f >= MIN_BALANCE; }
void deduct(long long f)    { balance -= f; }    // no check: can go negative
int main() {
   if (canTravel(1000)) deduct(1000);
   balance -= 99999;                          // legal here: the negative-balance bug
   std::cout << balance << '\n';
}
```

7.2 Object-oriented version

```
class SmartCard {
   long long balance_;                        // private
public:
   explicit SmartCard(long long b) : balance_{b} {}
   bool canTravel(long long f) const { return balance_ - f >= MIN_BALANCE_PAISE; }
   bool deduct(long long f) {
      if (f < 0 || !canTravel(f)) return false;
      balance_ -= f;
      return true;
   }
   bool recharge(long long amount) { if (amount <= 0) return false; balance_ += amount; return
true; }
   long long balance() const { return balance_; }
};
```

Test run (main creates cards with 10000, 6000 and 5999 paise):

Output:

```
1 9000      deduct(1000) from 10000 -> allowed, balance 9000
0 9000      deduct(99999) -> refused, balance unchanged
0 9000      recharge(-500) -> refused, balance unchanged
1 5000      6000 - 1000 = exactly Rs 50 -> allowed
0 5999      5999 - 1000 would leave 4999 -> refused
```

7.3 How the class prevents a negative balance, and which version is easier to change

The only code that changes balance_ is deduct() and recharge(), and deduct() refuses any fare that would leave less than Rs 50. Writing card.balance_ -= 5; from outside is a compile-time error. When the minimum balance changes, the object-oriented version needs one constant (or one line in canTravel) changed; the procedural version needs every place that touches balance reviewed. The object-oriented version is easier to change.

7.4 Alternative concept and complexity analysis: public data member

If balance were a public member, any code could write card.balance = -500;. **Time and space do not change** (O(1) and 8 bytes). But rule 1 (balance never below Rs 50) can now be broken by any part of the program, and so can rule 4 (exact, valid values), because the class can no longer guarantee its own invariant. A bug would have to be searched for in the whole program instead of in one class.

Task 8 - Read and Print Data Safely

8.1 Program

```
int readInt(const char* prompt) {
    int v;
    while (true) {
        std::cout << prompt;
        if (std::cin >> v) return v;                // success
        if (std::cin.eof()) return -1;              // input ended
        std::cin.clear();                          // clear the fail state
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // discard bad text
        std::cout << "Invalid number, try again.\n";
    }
}
void printMoney(long long paise) {
    std::cout << "Rs " << paise / 100 << '.' << std::setw(2) << std::setfill('0')
              << paise % 100 << std::setfill(' ');
}
```

8.2 Run with invalid input (letters typed first, then 9)

Output:

```
Hour (0-23): Invalid number, try again.
Hour (0-23): You entered hour 9
Fare: Rs 36.05
36.05
```

The operator typed abc and then 9. The first read failed; the program recovered and accepted 9. (Take a screenshot of this run for the GitHub folder.)

8.3 Discussion

- When letters are typed, cin >> v sets the fail bit; cin.fail() becomes true and every later read fails until the state is reset.
- cin.clear() resets the error state; cin.ignore(...) discards the bad characters. Without ignore, the loop would run forever on the same bad text.

- <iomanip> formatting: setw sets a field width, setfill the padding character, fixed with setprecision(2) prints exactly two decimals. Money is printed from integer paise as f / 100 and f % 100, so 3605 paise prints as Rs 36.05.
- Journey report layout: Journey no., Zone, Hour, Fare (Rs), Balance (Rs), each in a setw column.

8.4 Alternative concept and complexity analysis: getline with stoi in try/catch

```
std::string line;
std::getline(std::cin, line);
try { int v = std::stoi(line); /* use v */ }
catch (const std::exception&) { std::cout << "Invalid input\n"; }
```

Time: processing a line of length L is O(L) in both approaches. **Space:** getline needs a buffer of O(L) for the string; cin >> v reads directly with O(1) extra space. **Recovery:** getline removes the whole line, so no clear() or ignore() is needed, and stoi throws an exception on invalid input. But stoi("12abc") accepts 12 unless the number of characters parsed is checked, so extra validation is required. The getline method is more robust for line-based input; the cin >> method is shorter.

Task 9 - Implement and Test the Fare Logic

9.1 Implementation

```
constexpr long long BASE[] = {1000, 2000, 3000};
constexpr int PEAK_PERCENT = 20;
bool isPeak(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }
long long fare(int zoneIndex, int hour) {          // -1 means invalid
    if (zoneIndex < 0 || zoneIndex > 2 || hour < 0 || hour > 23) return -1;
    long long f = BASE[zoneIndex];
    if (isPeak(hour)) f += f * PEAK_PERCENT / 100;
    return f;
}
struct Test { int zone, hour; long long expected; };
// loop over a table of Test values, compare fare(zone, hour) with expected
```

9.2 Test table with expected and actual results

#	Case	Zone	Hour	Expected (paise)	Actual	Result
1	Off-peak, Inner	0	12	1000	1000	PASS
2	Off-peak, Middle	1	12	2000	2000	PASS
3	Off-peak, Outer	2	12	3000	3000	PASS
4	Peak, Inner	0	9	1200	1200	PASS
5	Peak, Middle	1	18	2400	2400	PASS
6	Peak, Outer	2	8	3600	3600	PASS
7	Just before peak	0	7	1000	1000	PASS
8	Peak start 8:00	0	8	1200	1200	PASS
9	Peak end 10:00 (off-peak)	0	10	1000	1000	PASS
10	Evening start 17:00	0	17	1200	1200	PASS
11	Evening end 20:00 (off-peak)	0	20	1000	1000	PASS
12	Unknown zone	5	9	-1 (rejected)	-1	PASS
13	Invalid hour	0	25	-1 (rejected)	-1	PASS

Program output: 13/13 passed (every row printed PASS).

9.3 Balance boundary and other invalid cases

- Balance exactly Rs 50 after the fare: 6000 - 1000 = 5000, allowed (>=). Balance 5999 - 1000 = 4999, denied. Both were run in the Task 7 program and gave the results shown there.
- Negative recharge: recharge(-500) returns false and the balance stays 9000 (Task 7 output).
- Letters typed for the hour: handled by readInt in Task 8, which rejects abc and asks again.
- Unknown zone and invalid hour: fare() returns -1 and the journey is skipped (tests 12 and 13).

9.4 Five journeys whose total is calculated by hand

Journey	Zone	Hour	Peak?	Calculation	Fare (paise)
1	Inner	12	No	1000	1000
2	Middle	9	Yes	2000 + 400	2400
3	Outer	15	No	3000	3000
4	Inner	18	Yes	1000 + 200	1200
5	Outer	10	No (10:00 off-peak)	3000	3000
Total				1000+2400+3000+1200+3000	10600 = Rs 106.00

9.5 Alternative concept and complexity analysis: separate hand-written checks

Testing with a table of test cases in a loop costs **O(t) time** for t tests and **O(t) space** for the table (13 rows here, a few hundred bytes). Adding a new boundary case is one new row of data. Hand-written checks cost the same O(t) time, but every new case needs new code, the checks are repetitive and it is easy to forget one. The table method is therefore easier to extend and to review.

Task 10 - Compare Expectations with Execution

10.1 Experiment

Each journey costs Rs 20.10 (2010 paise). The total is accumulated for 20,000, 500,000 and 3,000,000 journeys in double rupees, in exact long long paise, and in a 32-bit integer.

```
for (long long k = 0; k < n; ++k) { d += 20.10; p += 2010; u32 += 2010u; }
```

Output:

```
20000 journeys:  double=402000.000000      paise/100=402000.000000      diff=-0.000000
exact_paise=40200000     int32_wrapped=40200000  time_us=56
500000 journeys:  double=10049999.999905    paise/100=10050000.000000    diff=-0.000095
exact_paise=1005000000  int32_wrapped=1005000000 time_us=1460
3000000 journeys: double=60300000.003007    paise/100=60300000.000000    diff=0.003007
exact_paise=6030000000  int32_wrapped=1735032704 time_us=8545
```

(int32_wrapped is the 32-bit value; the program used an unsigned accumulator and converted it, to show the wrap without undefined behaviour. Run times are from my machine and will differ on yours.)

Journeys	**Exact total (paise)**	**double total (Rs)**	**Difference from exact**	**32-bit int**
20,000	40,200,000	402000.000000	about 0	correct
500,000	1,005,000,000	10049999.999905	-0.000095 Rs	correct (below 2,147,483,647)
3,000,000	6,030,000,000	60300000.003007	+0.003007 Rs	WRONG: 1,735,032,704 (overflow)

10.2 Do the totals agree? Does int overflow?

The integer-paise totals match the hand calculation exactly. The double totals differ by a tiny amount that grows with the number of journeys (zero at 20,000, -0.000095 Rs at 500,000, +0.003007 Rs at 3,000,000), because 20.10 cannot be represented exactly in binary. A 32-bit int works up to about 2.1 billion paise, so it is correct at 20,000 and 500,000 journeys but overflows at 3,000,000 (6,030,000,000 > 2,147,483,647) and gives a wrong number. (Signed overflow is undefined behaviour in C++, so the real result of a signed int is not guaranteed.)

10.3 Compiler warnings with -Wall -Wextra

- Task 4 produced: comparison of integer expressions of different signedness: int and unsigned int [-Wsign-compare]. It means a signed value is converted to unsigned in a comparison, which can give a wrong result for negative numbers. Fixed by comparing as long long or checking s < 0 first.
- The other programs compiled with no warnings. Other warnings you may see on your machine: unused variable or parameter (a name that is declared but never used) and conversion warnings when a double is stored into an int. Paste any warnings your own compiler printed and explain each.

10.4 Reasons for the differences

The difference between the expected and observed double totals comes from binary floating-point representation: most decimal fractions such as 0.10 or 20.10 have no exact binary form, so each addition rounds and the errors accumulate. The int overflow comes from the fixed 32-bit range. Both problems disappear with long long paise.

10.5 Alternative concept and complexity analysis: int, long long or double accumulator

Accumulator	**Result at 3,000,000 journeys**	**Failure kind**	**Time per addition**	**Space**
int (32-bit)	wrong (wraps)	overflow	O(1)	4 bytes
long long	exact 6,030,000,000 paise	none (limit about 9.2 x 10^18)	O(1)	8 bytes
double	close but off by 0.003 Rs	loss of precision	O(1)	8 bytes

All three take constant time per addition and only 4 or 8 bytes. The measured total time for 3,000,000 additions is a few milliseconds, so the choice is made by correctness, not speed: long long fails only at about 9.2 x 10^18 paise, far beyond the network total.

Task 11 - Recommend and Defend the Final Design

11.1 Recommended design

- **Types:** long long paise for all money and totals, int for hours and percentages, bool for flags, enum class Zone.
- **Constants:** every fare, surcharge, discount and minimum balance as constexpr in one header file.
- **Classes:** SmartCard (private balance, canTravel, deduct, recharge), a fare function or FareTable, and Pass.
- **Evidence from my experiments:** Task 3 and 10 show that double drifts (+0.003007 Rs at 3,000,000 journeys) and that a 32-bit int overflows at the network size; Task 9 shows all 13 boundary and invalid tests pass with half-open peak intervals; Task 7 shows the private balance cannot be driven below Rs 50 or made negative.

11.2 How a fare revision is carried out

Edit the one constant (for example BASE_PAISE) in the constants header, recompile with g++ -std=c++17 -Wall -Wextra, rerun the Task 9 test table to confirm every row still passes (updating the expected values for the changed fare), and redeploy the gate software.

11.3 Limitations

- A recompile and redeploy is needed for each fare revision.
- The concession percentages are assumed, not given by the assignment.
- Invalid input is reported by returning -1 instead of throwing an exception.
- There is no storage of the card between runs and no protection against two gates using the same card at the same time (this would need locking).

11.4 Is the design correct for the whole network?

Yes. Each journey is O(1) work and the total for n journeys is O(n). The network total of 6.03 x 10^9 paise (3,000,000 journeys at Rs 20.10) is far below the long long limit of about 9.2 x 10^18, and the measured integer total was exact. The int-based version would fail, which is why long long is recommended.

11.5 Alternative concept and complexity analysis: fare table read from a configuration file

```
std::ifstream in("fares.txt");
long long fare[4]; for (int i = 0; i < 4 && in >> fare[i]; ++i) {}
```

Start-up time: reading z fare values is O(z) (microseconds for four zones). **Space:** O(z) for the values (4 x 8 = 32 bytes). **Benefit:** a fare revision needs only an edited text file, no recompile and no redeploy. **Costs:** extra code to open, parse and validate the file; the risk of a missing, corrupt or wrongly edited file; and the values are no longer compile-time constants, so they cannot be constexpr or switch labels. **Conclusion:** for a small, rarely changing table the compiled-in constexpr table is simpler and safer; if fares change often, a validated configuration file is worth its small start-up cost.

Task 12 - Day-4 Design Challenge: Re-evaluate and Adapt the Design

Changed requirement: a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass giving unlimited journeys in two chosen zones. **Decision: MODIFY** the original design.

A. Impact Analysis (2 marks)

Element	Affected?	What changes
BASE_PAISE[] / fare table	Yes	Add 6000 for Airport
enum class Zone	Yes	Add Airport
Every switch on the zone	Yes	Add case Zone::Airport (easy to miss one)
Fare expressions	Yes	Add a weekend branch checked before the peak branch
Constants	Yes	Add WEEKEND_DISCOUNT_PERCENT = 10
SmartCard class	Small	Balance rules unchanged; needs an optional pass link
Pass class	New	Holds two zones; covers(zone)

B. Evaluation of the Original Design (2 marks)

Worked examples with the new rules: Airport weekday 12:00 = 6000. Airport weekday 09:00 (peak) = 6000 + 1200 = 7200. Airport weekend 09:00 = 6000 - 600 = 5400 (not 7200, because the discount replaces the surcharge). Inner weekend 12:00 = 1000 - 100 = 900.

Switch-based fare logic: it can absorb the new zone but with one edit to each switch; a forgotten switch silently returns a fare of 0 for Airport. The weekend rule is only an extra branch, but it must be placed before the peak check or the two rules would combine incorrectly.
SmartCard class: it absorbs the change well, because the business rule (balance at least Rs 50 after payment) does not change. A pass is a separate concept, so putting the zone logic inside SmartCard would mix two responsibilities; a separate Pass class is better. The overall structure (constexpr constants, long long paise, private balance) is sound, so a full redesign is not justified.

C. Adaptation (2 marks) - code

```
enum class Zone { Inner, Middle, Outer, Airport };
```

```
constexpr std::array<long long, 4> FARE_PAISE{1000, 2000, 3000, 6000};  // indexed by zone
constexpr int PEAK_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }

long long fare(Zone z, int hour, bool weekend) {
    long long f = FARE_PAISE[static_cast<std::size_t>(z)];
    if (weekend)          f -= f * WEEKEND_DISCOUNT_PERCENT / 100;  // never combined
with peak
    else if (isPeak(hour)) f += f * PEAK_PERCENT / 100;
    return f;
}

class Pass {                          // monthly pass: two chosen zones
    Zone a_, b_;
public:
    Pass(Zone x, Zone y) : a_{x}, b_{y} {}
    bool covers(Zone z) const { return z == a_ || z == b_; }
};

class SmartCard {
    long long balance_;
    const Pass* pass_ = nullptr;
public:
    explicit SmartCard(long long b) : balance_{b} {}
    void setPass(const Pass* p) { pass_ = p; }
    bool canTravel(long long f) const { return balance_ - f >= MIN_BALANCE_PAISE; }
    bool travel(Zone z, int hour, bool weekend) {
        if (pass_ && pass_->covers(z)) return true;     // unlimited: no deduction
        long long f = fare(z, hour, weekend);
        if (!canTravel(f)) return false;
        balance_ -= f;
        return true;
    }
```

```
    long long balance() const { return balance_; }
};
```

The fare table is now indexed by the zone, so a fifth zone is one more array element and one more enum value, with no switch to update. A static_assert on the array size keeps the enum and the table in step.

D. Evidence-Based Defence (2 marks)

Program output (all compiled and run):

Output:

```
Airport weekday 12:00 = 6000
Airport weekday 09:00 = 7200
Airport weekend 09:00 = 5400
Inner weekend 12:00  = 900
Balance after pass journey = 20000
Balance after non-pass Outer journey = 17000
Exactly Rs 50 left allowed: 1 balance 5000
```

Test	Expected	Actual	Result
Airport weekday 12:00	6000	6000	PASS
Airport weekday 09:00 (peak)	7200	7200	PASS
Airport weekend 09:00	5400 (no surcharge)	5400	PASS
Inner weekend 12:00	900	900	PASS
Journey inside pass zones	balance unchanged (20000)	20000	PASS
Journey outside pass zones	fare deducted (20000 - 3000)	17000	PASS
Balance exactly Rs 50 after fare	allowed	allowed, balance 5000	PASS

Defence of MODIFY: the rules that matter, exact paise arithmetic and the Rs 50 minimum balance, did not change, and the tests above show they still hold. The change cost one array element, one enum value, one constant, one branch and one small class. A full redesign would add risk with no benefit. **Trade-offs:** the array lookup removes the per-switch edits and is O(1) with 4 x 8 = 32 bytes, but it depends on the enum order matching the array order, so a

static_assert is used. Keeping the Pass outside the balance logic keeps each class to a single responsibility, at the cost of one extra pointer per card. **Weekend and peak:** an if / else-if keeps the two discounts from combining, as required.

Appendix - GitHub Submission Guide (what you need to do)

The assignment will not be evaluated without a working GitHub link, and the commit history is checked, so commit progressively, at least once for each task.

Step 1: Create the repository

- Go to github.com and create a new repository named exactly: DSA01-<RegisterNumber>-Assignment1 (for example DSA01-192011234-Assignment1).
- Make it public, or private and add the course faculty as a collaborator (Settings, Collaborators). Tick "Add a README".

Step 2: Set up on your computer

```
git clone https://github.com/<your-username>/DSA01-<RegisterNumber>-Assignment1.git
cd DSA01-<RegisterNumber>-Assignment1
git config user.name  "Your Name"
git config user.email "you@example.com"
mkdir Task01 Task02 Task03 Task04 Task05 Task06 Task07 Task08 Task09 Task10 Task11
Task12
```

Step 3: Put these files in each folder

Folder	Files to add	What to run
Task01	task01_analysis.md (objects, UML, rules), UML image	no program
Task02	constants.hpp, task02.cpp	g++ -std=c++17 -Wall -Wextra task02.cpp -o task02
Task03	task03.cpp (sizeof, climits, double vs paise), screenshot	compile and run, save output
Task04	task04.cpp (the given program) + task04_fixed.cpp, predicted output file, screenshot	run; write the prediction BEFORE running
Task05	task05.cpp (the given program), predicted output file, screenshot	run; write the prediction BEFORE running
Task06	task06.cpp (completed TODO 1 and 2), pseudocode file, screenshot	run and compare with the expected output

Folder	Files to add	What to run
Task07	task07_procedural.cpp, task07_oop.cpp, screenshots	run both
Task08	task08.cpp, input.txt (with letters then numbers), screenshot	run with abc then 9
Task09	task09.cpp (test table), screenshot of 13/13 passed	run
Task10	task10.cpp, screenshot, warnings text	run and copy warnings
Task11	task11_recommendation.md, optional config-file demo	optional run
Task12	task12.cpp, screenshot of the output	run

Step 4: Compile and capture a screenshot for every program

```
g++ -std=c++17 -Wall -Wextra task06.cpp -o task06
./task06          # on Windows: task06.exe
```

Take a screenshot of the terminal showing the command and its output, and save it in the same folder (for example task06_output.png). Do not use my outputs as your screenshots: every student must execute every task individually.

Step 5: Commit after each task (at least one commit per task)

```
git add Task01
git commit -m "Task 1: OO analysis, objects and UML"
git add Task02
git commit -m "Task 2: constants, tokens, enum class"
git add Task03
git commit -m "Task 3: data types, sizeof, paise vs double experiment"
# ...continue the same way for Task04 to Task12...
git add Task12
git commit -m "Task 12: Airport zone, weekend discount, Pass class"
git push origin main
```

Push after each commit or after every few. Do the commits on different days or at different times as you actually finish each task; a single upload of everything on the last day is not accepted.

Step 6: README.md (in the repository root)

```
# DSA01 - Assignment 1: Namma Metro Smart-Card Fare System
Name: <your name>
Register number: <your register number>
Compiler: g++ <your version> (check with: g++ --version)
Compile command: g++ -std=c++17 -Wall -Wextra <file>.cpp -o <file>

## How to run
cd TaskNN
g++ -std=c++17 -Wall -Wextra <file>.cpp -o <file>
./<file>

## Results
Task 1: <one line>
Task 2: <one line>
...
Task 12: <one line>
```

Step 7: Finish the report

1. Write your name, register number, department and the GitHub link on the cover page.
2. Replace the screenshots you cannot attach here with your own; add your measured timings if they differ from the ones shown.
3. Write your own predicted outputs for Tasks 4 and 5 before running the programs, and explain any difference.
4. Check the concession percentages (Student 50%, Senior 30%) against your course notes and change them if they differ.
5. Export the document to PDF if your faculty asks for it, and check that the GitHub link opens when you are logged out (for a public repository) or that the faculty member is added as a collaborator.

Step 8: Prepare for the viva

You may be asked to explain, execute or modify any part of the code. Be ready to explain: why paise are used instead of double; why peak uses a half-open interval; how SmartCard prevents

a negative balance; why the Task 4 comparison s < u is false; how the Task 12 design absorbs the new zone; and why int overflows at 3,000,000 journeys.